

Modules

What is a Module?

A module is a file same as a code library.

A file containing a set of functions you want to include in your application.

Create a Module

To create a module just save the code you want in a file with the file extension .py:

Save this code in a file named mymodule.py

```
def greeting(name):  
    print("Hello, " + name)
```

Use a Module

Now we can use the module we just created, by using the import statement:

Example

Import the module named mymodule, and call the greeting function:

1)

```
import mymodule
```

```
mymodule.greeting("Python")
```

2)

```
import mymodule as mm
```

```
mm.greeting("Python")
```

3)

```
from my module import greeting           (also can use *)
```

```
greeting("Python")
```

Variables in Module

The module can contain functions, as already described, but also variables of all types (arrays, dictionaries, objects etc):

Example

Save this code in the file mymodule.py

```
person1 = {  
    "name": "ABC",  
    "age": 26,  
    "country": "India"  
}
```

Example

Import the module named mymodule, and access the person1 dictionary:

```
import mymodule  
  
a = mymodule.person1["age"]  
print(a)
```

Built-in Modules

There are several built-in modules in Python, which you can import

Example

Import and use the platform module:

```
import platform
```

```
x = platform.system()
```

```
print(x)
```

Using the dir() Function

There is a built-in function to list all the function names (or variable names) in a module. The dir() function:

Example

List all the defined names belonging to the platform module:

```
import platform
```

```
x = dir(platform)
```

```
print(x)
```

Python math Module

Python has a built-in module that you can use for mathematical tasks.

The math module has a set of methods and constants.

Math Methods

Method	Description
<code>math.acos()</code>	Returns the arc cosine of a number
<code>math.acosh()</code>	Returns the inverse hyperbolic cosine of a number
<code>math.asin()</code>	Returns the arc sine of a number
<code>math.asinh()</code>	Returns the inverse hyperbolic sine of a number
<code>math.atan()</code>	Returns the arc tangent of a number in radians
<code>math.atan2()</code>	Returns the arc tangent of y/x in radians
<code>math.atanh()</code>	Returns the inverse hyperbolic tangent of a number
<code>math.ceil()</code>	Rounds a number up to the nearest integer
<code>math.comb()</code>	Returns the number of ways to choose k items from n items without repetition and order
<code>math.copysign()</code>	Returns a float consisting of the value of the first parameter and the sign of the second parameter
<code>math.cos()</code>	Returns the cosine of a number
<code>math.cosh()</code>	Returns the hyperbolic cosine of a number
<code>math.degrees()</code>	Converts an angle from radians to degrees
<code>math.dist()</code>	Returns the Euclidean distance between two points (p and q), where p and q are the coordinates of that point

<code>math.erf()</code>	Returns the error function of a number
<code>math.erfc()</code>	Returns the complementary error function of a number
<code>math.exp()</code>	Returns E raised to the power of x
<code>math.expm1()</code>	Returns $E^x - 1$
<code>math.fabs()</code>	Returns the absolute value of a number
<code>math.factorial()</code>	Returns the factorial of a number
<code>math.floor()</code>	Rounds a number down to the nearest integer
<code>math.fmod()</code>	Returns the remainder of x/y
<code>math.frexp()</code>	Returns the mantissa and the exponent, of a specified number
<code>math.fsum()</code>	Returns the sum of all items in any iterable (tuples, arrays, lists, etc.)
<code>math.gamma()</code>	Returns the gamma function at x
<code>math.gcd()</code>	Returns the greatest common divisor of two integers
<code>math.hypot()</code>	Returns the Euclidean norm
<code>math.isclose()</code>	Checks whether two values are close to each other, or not
<code>math.isfinite()</code>	Checks whether a number is finite or not
<code>math.isinf()</code>	Checks whether a number is infinite or not
<code>math.isnan()</code>	Checks whether a value is NaN (not a number) or not
<code>math.isqrt()</code>	Rounds a square root number downwards to the nearest integer

<code>math.ldexp()</code>	Returns the inverse of <code>math.frexp()</code> which is $x * (2^{**i})$ of the given numbers <code>x</code> and <code>i</code>
<code>math.lgamma()</code>	Returns the log gamma value of <code>x</code>
<code>math.log()</code>	Returns the natural logarithm of a number, or the logarithm of number to base
<code>math.log10()</code>	Returns the base-10 logarithm of <code>x</code>
<code>math.log1p()</code>	Returns the natural logarithm of $1+x$
<code>math.log2()</code>	Returns the base-2 logarithm of <code>x</code>
<code>math.perm()</code>	Returns the number of ways to choose <code>k</code> items from <code>n</code> items with order and without repetition
<code>math.pow()</code>	Returns the value of <code>x</code> to the power of <code>y</code>
<code>math.prod()</code>	Returns the product of all the elements in an iterable
<code>math.radians()</code>	Converts a degree value into radians
<code>math.remainder()</code>	Returns the closest value that can make numerator completely divisible by the denominator
<code>math.sin()</code>	Returns the sine of a number
<code>math.sinh()</code>	Returns the hyperbolic sine of a number
<code>math.sqrt()</code>	Returns the square root of a number
<code>math.tan()</code>	Returns the tangent of a number
<code>math.tanh()</code>	Returns the hyperbolic tangent of a number
<code>math.trunc()</code>	Returns the truncated integer parts of a number

Example

1)

```
import math

print(math.acos(0.55))

print(math.acos(-0.55))

print(math.acos(0))

print(math.acos(1))

print(math.acos(-1))
```

2)

```
import math

print (math.acosh(7))

print (math.acosh(56))

print (math.acosh(2.45))

print (math.acosh(1))
```

3)

```
import math
```



```
print(math.ceil(1.4))
```

```
print(math.ceil(5.3))
```

```
print(math.ceil(-5.3))
```

```
print(math.ceil(22.6))
```

```
print(math.ceil(10.0))
```

4)

```
import math
```

```
# Initialize the number of items to choose from
```

```
n = 7
```

```
# Initialize the number of possibilities to choose
```

```
k = 5
```

```
# Print total number of possible combinations
```

```
print (math.comb(n, k))
```

5)

```
import math
```

```
print(math.factorial(9))
```

```
print(math.factorial(6))
```

```
print(math.factorial(12))
```

6)

```
import math
```

```
print(math.pow(9, 3))
```

7)

```
import math
```

```
print(math.trunc(2.77))
```

```
print(math.trunc(8.32))
```

```
print(math.trunc(-99.29))
```

8)

```
import math
```

```
# Return the remainder of x/y
```

```
print (math.remainder(9, 2))
```

```
print (math.remainder(9, 3))
```

```
print (math.remainder(18, 4))
```

Math Constants

Constant	Description
-----------------	--------------------

math.e	Returns Euler's number (2.7182...)
--------	------------------------------------

math.inf	Returns a floating-point positive infinity
----------	--

math.nan	Returns a floating-point NaN (Not a Number) value
----------	---

math.pi	Returns PI (3.1415...)
---------	------------------------

math.tau	Returns tau (6.2831...)
----------	-------------------------

Example

1)

```
import math
```

```
# Print the value of E
```

```
print (math.e)
```

2)

```
import math
```

```
print (math.inf)
```

```
print (-math.inf)
```

3)

```
import math
```

```
print (math.nan)
```

4)

```
import math
```

```
print (math.pi)
```

5)

```
import math
```

```
print (math.tau)
```

Date Time Module

A date in Python is not a data type of its own, but we can import a module named `datetime` to work with dates as date objects.

Example

1)

```
import datetime
```

```
x = datetime.datetime.now()
```

```
print(x)
```

2)

```
import datetime
```

```
x = datetime.datetime.now()
```

```
print(x.year)
```

```
print(x.month)
```

```
print(x.day)
```

Creating Date Objects

To create a date, we can use the `datetime()`.

The `datetime()` class requires three parameters to create a date: year, month, day.

Example

```
import datetime
```

```
x = datetime.datetime(2020, 5, 17)
```

```
print(x)
```

Directive	Description	Example
%a	Weekday, short version	Wed
%A	Weekday, full version	Wednesday
%w	Weekday as a number 0-6	3
%d	Day of month 01-31	31
%b	Month name, short version	Dec
%B	Month name, full version	December
%m	Month as a number 01-12	12

%y	Year, short version	18
%Y	Year, full version	2018
%H	Hour 00-23	17
%I	Hour 00-12	5
%p	AM/PM	PM
%M	Minute 00-59	41
%S	Second 00-59	8
%f	Microsecond	548513
%z	UTC offset	100
%Z	Timezone	IST
%j	Day number of year 001-366	365
%U	Week number of year,	52
%W	Week number of year,	52
%c	Local version of date and time	Mon Dec 31 17:41:00 2018
%C	Century	20
%x	Local version of date	12/31/18
%X	Local version of time	17:41:00
%%	A % character	%

The strftime() Method

The datetime object has a method for formatting date objects into readable strings.

The method is called `strftime()`, and takes one parameter, `format`, to specify the format of the returned string:

Example

Display the name of the month:

```
import datetime
```

```
x = datetime.datetime.now()
```

```
print(x.strftime("%B"))
```

Random Module

Random number guessing game

```
import random
```

```
cnum = random.randrange(0,100)
```

```
unum = int(input("Enter Your Number: "))  
  
if cnum > unum:  
    print("Computer number ", cnum, " Is greater")  
  
elif unum > cnum:  
    print("Computer number ", cnum, " Is Smaller")  
  
else:  
    print("Computer number ", cnum, " Is equal")
```

Utility Programs

Example 1: Bulk File Rename

```
import os
os.chdir('D:/try')
i=1
for file in os.listdir():
    src = file
    dst = 'Image' + str(i) + '.jpg'
    os.rename(src, dst)
    i+=1
```

Example 2: Generate password in Python

```
import secrets
import string
alphabet = string.ascii_letters + string.digits

# for a 20-character password
password = "".join(secrets.choice(alphabet) for i in range(20)) print(password)
```

Example 3: QR code encoder

```
>>> pip install qrcode

import qrcode

data = 'https://www.coplur.com/home'

img = qrcode.make(data)

img.save('qr1.png')
```

Example 4: QR code decoder

```
pip install opencv-python

import cv2

image = cv2.imread("qr1.png")
detector = cv2.QRCodeDetector()
data, vertices_array, binary_qrcode = detector.detectAndDecode(image)

if vertices_array is not None and data:
    print(f"QR Data: {data}")
else:
    print("No Data Found.")
```

Example 5: Sudoku Solver using numpy

```
import numpy as np
```

```
puzzle = [  
    [0,4,8,0,0,0,0,1,6],  
    [5,0,0,0,9,2,0,0,8],  
    [0,0,0,1,0,0,0,0,0],  
    [9,0,0,3,0,0,8,2,0],  
    [0,2,0,0,0,0,0,7,0],  
    [0,8,4,0,0,6,0,0,5],  
    [0,0,0,0,0,3,0,0,0],  
    [4,0,0,9,6,0,0,0,2],  
    [1,6,0,0,2,0,7,0,0]  
]
```

```
def is_valid(row_index, column_index, number):
```

```
    global puzzle
```

```
    for n in range(0,9):
```

```
        if puzzle[row_index][n] == number:
```

```
            return False
```

```
    for n in range(0,9):
```

```
        if puzzle[n][column_index] == number:
```

```
            return False
```

```
square_col = (column_index // 3) * 3
square_row = (row_index // 3) * 3
for n in range(0,3):
    for m in range(0,3):
        if puzzle[square_row+n][square_col+m] == number:
            return False

return True
```

```
def solve():
    global puzzle
    for row in range(0,9):
        for column in range(0,9):
            if puzzle[row][column] == 0:
                for number in (range(1,10)):
                    if is_valid(row, column, number):
                        puzzle[row][column] = number
                        solve()
                        puzzle[row][column] = 0
                return
```

```
print(np.matrix(puzzle))
return
```

```
solve()
```